

查询改写、多路召回与重排序

DOC_ID: DOC-RAG-ADV-001

说明：本文档为 RAGLab 项目生成的教学用合成语料，用于分块、检索、重排、引用和评测实验。它不是 LangChain/LangGraph 官方文档，也不应被当作 API 版本说明。文中的稳定标识（DOC_ID、SECTION_ID）用于构建可重复的检索评测集。

1. 为什么原始查询不一定适合检索

SECTION_ID: ADV-QUERY-01

用户问题往往面向对话，而检索器只看到当前字符串。例如用户追问“那它保存在什么地方”，其中“它”依赖历史上下文。直接检索会丢失实体。

查询改写的目标是生成一个独立、可检索、尽量不扩展原意的问题。例如：

原始追问：那它保存在什么地方？

改写后：LangGraph Checkpointer 将线程执行状态持久化在哪里？

改写错误会把整个检索带偏，因此必须记录原始查询和改写查询，并允许评测二者差异。

2. Multi-Query Retrieval

SECTION_ID: ADV-MULTIQ-01

Multi-Query 为同一意图生成多个表达，再分别检索并去重。它可以提高召回，但也会增加调用次数和噪声。

例如“RAG 为什么出现正确文档却答错”可以产生：

1. 检索正确但生成错误的原因；
2. RAG 忠实性失败；
3. 上下文已包含答案但模型未引用。

多查询适合概念性问题，不一定适合精确 ID 查询。

3. HyDE

SECTION_ID: ADV-HYDE-01

HyDE 先让模型生成一个假设性答案，再用该答案做向量检索。它利用答案风格文本与文档语料的相似性提高语义召回。

风险是模型的假设可能错误，导致检索方向偏移。因此 HyDE 应作为候选策略，而不是默认对所有问题启用。

4. Reranker 的作用

SECTION_ID: ADV-RERANK-01

第一阶段 Retriever 追求召回，Reranker 负责更精细地判断“这个文档块是否能回答这个问题”。典型做法是先召回 20 个候选，再重排选出 4 到 6 个上下文块。

Reranker 可以使用交叉编码器或 LLM，但后者成本和延迟更高。重排后的分数不能与原始向量相似度直接混用，应分别记录。

5. Context Compression

SECTION_ID: ADV-COMPRESS-01

上下文压缩不是简单截断，而是从召回块中抽取与问题直接相关的句子。它可以减少 Token，但也可能删除限定条件、否定词和上下文关系。

对于政策、法律和技术限制类文档，应谨慎使用抽取式压缩，并保留原始块以便引用校验。

6. 检索充分性判断

SECTION_ID: ADV-SUFF-01

Hybrid RAG 可以在生成前判断文档是否足够。如果检索结果与问题无关，系统应改写查询或明确回答证据不足，而不是强行生成。

充分性判断可以结合：

- 最高重排分数；
- 相关块数量；
- 必要实体是否出现；
- LLM 的结构化判断；
- 问题类型对应的最低证据规则。

7. 策略路由

SECTION_ID: ADV-ROUTE-01

不同问题适合不同检索策略：

问题类型	推荐策略
精确类名、错误码	BM25 + metadata filter
语义解释	Dense 或 Hybrid
对话追问	历史感知改写 + Hybrid
跨章节比较	Multi-Query + Rerank
文档外问题	充分性判断后拒绝

策略路由的价值是控制成本，而不是让每个问题都执行最复杂的流水线。